



Use of Generative AI: Prompt and Response Log

*R&D Project: Peer-to-Peer Directory Synchronization
with Lightweight Version Control*

Arihant Jain

Su Huizhe

Nizar Soufiya

Shengkang Duan

EPITA Graduate School of Computer Science

Advisor

Thomas Broussard

June 22, 2026

Declaration and Citation of Generative AI

In accordance with EPITA's academic standards and the Action Learning programme guidelines, we declare that we utilized generative AI (specifically OpenAI's ChatGPT and Google's Gemini) during the research, design, and technical drafting phases of our peer-to-peer file synchronization system. Our utilization was focused strictly on clarifying abstract distributed systems paradigms, structuring protocol formats, and formulating logical data boundaries. The final implementation, architecture choices, and written arguments in the thesis are the original work of the authors.

As instructed, the tools are referenced in the thesis bibliography under APA style guidelines. To ensure complete transparency and allow the reviewer to assess the quality of our prompting, this document logs the specific queries and generated outputs that directly informed the technical design of our system.

APA Style Citations

In-Text Citation Format

We referenced the logical clocks design using OpenAI (2026), generated the readability parsing scripts using ChatGPT (OpenAI, 2026), and structured the frontend layout based on guidance from Gemini (Google, 2026).

Bibliographic Entries

Google. (2026). Gemini (Gemini 1.5 Pro version) [Large language model]. <https://gemini.google.com>. (Accessed: 2026-06-22)

OpenAI. (2026). ChatGPT (GPT-4o version) [Large language model]. <https://chatgpt.com>. (Accessed: 2026-06-22)

Chapter 1

Log of Prompts and Responses

This chapter details the specific prompt logs and responses that assisted in the R&D project.

1.1 Last-Write-Wins (LWW) and Local Rollback Versions (C++)

This conversation was held to refine our conflict resolution policy. We settled on Last-Write-Wins for its ease of implementation on temporary student networks. However, to mitigate the risk of accidental overwrites (data loss), we used ChatGPT to outline a safe C++ file-watching rollback system.

Prompt by Arihant Jain:

We are designing a P2P directory synchronization system for 3 to 5 peers on campus Wi-Fi. We chose Last-Write-Wins (LWW) with local backups to handle conflict resolution, as consistency is sacrificed for availability under the CAP theorem. How should we represent logical clock timestamps in the file metadata, and how can we implement the rollback versioning directory in C++ to prevent permanent data loss during LWW overwrites?

Response by ChatGPT:

To implement safe LWW synchronization with rollback support, you can establish the following structure:

1. **File Metadata Schema:** Each file should carry a logical timestamp (Lamport clock counter) and a content hash (SHA-256). For example:

```
struct FileMetadata {  
    std::string filepath;  
    uint64_t lamport_clock;
```

```

    std::string sha256_hash;
    uint64_t modified_time; // Physical timestamp for tie-breaking
};

```

2. **C++ Safety Rollback Function:** Before overwriting a local file during a sync update, move the existing local version to a hidden backup directory (.versions/) and append a unique timestamp suffix to it. This ensures that if the incoming update is a corrupt edit, the original file is preserved and can be manually recovered.

Here is a C++ implementation outline using `std::filesystem`:

```

#include <filesystem>
#include <string>
#include <chrono>

namespace fs = std::filesystem;

bool backup_and_overwrite(const std::string& target_path,
                          const std::string& temp_incoming_path) {
    try {
        if (fs::exists(target_path)) {
            fs::path orig(target_path);
            fs::path backup_dir = orig.parent_path() / ".versions";
            if (!fs::exists(backup_dir)) {
                fs::create_directories(backup_dir);
            }
            auto dur = std::chrono::system_clock::now()
                      .time_since_epoch();
            auto now = std::chrono::duration_cast<
                std::chrono::milliseconds>(dur).count();
            std::string backup_name = orig.stem().string() + "_" +
                                      std::to_string(now) +
                                      orig.extension().string();
            fs::copy_file(target_path, backup_dir / backup_name);
        }
        fs::copy_file(temp_incoming_path, target_path,
                      fs::copy_options::overwrite_existing);
        return true;
    } catch (const fs::filesystem_error& e) {
        return false;
    }
}

```

1.2 Peer Discovery via mDNS and IPC (Go)

This prompt helped us design the discovery architecture. We used Go for peer discovery since libraries for mDNS/DNS-SD are mature and concurrent programming is clean. We asked ChatGPT to outline how the Go daemon should notify the C++ engine when a peer is discovered.

Prompt by Su Huizhe:

For peer discovery on a local campus Wi-Fi subnet, we are using mDNS and DNS-SD in Go. We need to broadcast a service under the ‘_p2psync._tcp’ domain and dynamically discover other peers. How can we implement the discovery daemon in Go using the ‘zeroconf’ library, and how should we coordinate the discovery events to trigger direct TCP connections to the resolved addresses?

Response by ChatGPT:

You can achieve this in Go using the github.com/grandcat/zeroconf library. The logic splits into service registration and service browsing.

Here is a Go skeleton showing the registration and browser loop:

```
package main

import (
    "context"
    "log"
    "time"
    "github.com/grandcat/zeroconf"
)

func startMDNS(ctx context.Context, peerID string, port int,
               discoveredChan chan<- string) {
    // Register service
    server, err := zeroconf.Register(peerID, "_p2psync._tcp", "local.",
                                     port, []string{"txtv=0"}, nil)
    if err != nil {
        log.Fatalf("Registration error: %v", err)
    }
    defer server.Shutdown()

    // Browse service
    resolver, err := zeroconf.NewResolver(nil)
    if err != nil {
        log.Fatalf("Resolver error: %v", err)
    }
}
```

```
}

entries := make(chan *zeroconf.ServiceEntry)
go func() {
    for entry := range entries {
        if entry.Instance != peerID {
            addr := entry.AddrIPv4[0].String() + ":" +
                string(entry.Port)
            discoveredChan <- addr
        }
    }
}()

err = resolver.Browse(ctx, "_p2psync._tcp", "local.", entries)
if err != nil {
    log.Fatalf("Browse error: %v", err)
}
<-ctx.Done()
}
```

When `discoveredChan` receives an entry, the Go connection manager dials a TCP connection. Once established, Go notifies C++ via the Unix domain socket IPC using a JSON event payload.

1.3 Readability Metric Calculator (Python)

This conversation was held to build the readability calculation tool. To comply with the Flesch Readability guidelines, we needed a Python script to strip LaTeX markup from the thesis source files and run the standard formulas.

Prompt by Nizar Soufiya:

As part of our thesis guidelines, we need to calculate the Flesch Reading Ease and Flesch-Kincaid Grade Level for our LaTeX chapters. We need a Python script that reads our LaTeX files, strips out LaTeX command tags and mathematical environments, and computes these metrics using the standard formulas. How should we implement this parser in Python?

Response by ChatGPT:

You can achieve this by writing a Python script that uses regular expressions to clean the text, and then parses the words to count syllables.

Here is the code skeleton:

```
import re

def clean_latex(text):
    # Remove comments
    text = re.sub(r'%.*\n', '', text)
    # Remove inline math
    text = re.sub(r'\$.*?\$', '', text)
    # Remove LaTeX commands: \command{args}
    text = re.sub(r'\\[a-zA-Z]+\{([^\}]+\)\}', r'\1', text)
    # Remove raw commands: \command
    text = re.sub(r'\\[a-zA-Z]+', '', text)
    return text.strip()

def count_syllables_word(word):
    word = word.lower()
    count = 0
    vowels = "aeiouy"
    if len(word) == 0: return 0
    if word[0] in vowels: count += 1
    for index in range(1, len(word)):
        if word[index] in vowels and word[index - 1] not in vowels:
            count += 1
    if word.endswith("e"): count -= 1
    if count == 0: count = 1
```

```
    return count

def calculate_flesch(sentences, words, syllables):
    w_s = words / sentences
    s_w = syllables / words
    ease = 206.835 - 1.015 * w_s - 84.6 * s_w
    grade = 0.39 * w_s + 11.8 * s_w - 15.59
    return ease, grade
```

1.4 JavaFX Frontend Project Structure (Gemini)

This conversation was held to define a clean project structure for our JavaFX user interface. Since the UI functions strictly as a frontend GUI that displays synchronization states and conflict logs, it relies on IPC sockets to communicate with the local Go daemon. We queried Gemini to structure a clean, strictly-frontend Maven template.

Prompt by Nizar Soufiya:

I am developing a JavaFX project that acts strictly as the frontend GUI for a peer-to-peer file synchronization system. It does not need to handle database connections or network sockets itself; it will communicate with a local daemon. What is the clean, recommended project structure for a strictly frontend JavaFX project using Maven and FXML

Response by Gemini:

A clean structure for a frontend-only JavaFX application keeps the UI layout (FXML), styles (CSS), and logic (Controllers) strictly decoupled, and uses a dedicated IPC client module to bridge communication with the backend daemon.

Here is the recommended Maven project structure:

```
p2p-gui/
|-- pom.xml
'-- src/
    '-- main/
        |-- java/
        |   '-- p2p/
        |       '-- gui/
        |           |-- App.java                # Entry point
        |           |-- controller/
        |               |-- MainController.java # Main ctrl
        |               '-- PeersController.java # Peer ctrl
        |           '-- ipc/
        |               '-- IpcClient.java      # IPC client
        '-- resources/
            '-- p2p/
                '-- gui/
                    |-- style.css                # Styling
                    '-- view/
                        |-- main.fxml            # Main view
                        '-- peers.fxml           # Peer view
```

Key architectural points for this frontend structure:

1. **Decoupled FXML and Controllers:** Controllers reside under the `controller/` package, while FXML files are under `resources/p2p/gui/view/`. In each FXML file, specify the controller class via the `fx:controller` attribute.
2. **Daemon Interface (IPC client):** The `IpcClient` class runs in a background thread using JavaFX concurrency utilities (like `Task` or `Service`). It reads status updates from the backend socket and uses `Platform.runLater()` to safely update JavaFX UI elements with peer and file listings.